BingePlay — Complete SQL Report

The Unlox Academy Minor Project · Advanced SQL · Week 4

This document contains the MySQL Workbench SQL solutions for all 12 business questions from the BingePlay project brief, including the required queries and brief explanations.

Database Setup & Verification

```
USE bingeplay;

SELECT 'users' AS tbl, COUNT(*) AS row_count FROM users
UNION ALL
SELECT 'subscriptions', COUNT(*) FROM subscriptions
UNION ALL
SELECT 'shows', COUNT(*) FROM shows
UNION ALL
SELECT 'watch_sessions', COUNT(*) FROM watch_sessions
UNION ALL
SELECT 'ratings', COUNT(*) FROM ratings;

SELECT COUNT(*) AS null_user_ids
FROM watch_sessions
WHERE user_id IS NULL;
```

Q1 — Active Revenue

```
SELECT
    COUNT(*) AS active_subscriptions,
    SUM(monthly_price_inr) AS total_monthly_revenue
FROM subscriptions
WHERE status = 'active'
  AND (end_date IS NULL OR end_date > '2024-06-30');
```

Q2 — Signup Momentum

```
SELECT
    MONTH(signup_date) AS month_number,
    MONTHNAME(signup_date) AS month,
    COUNT(*) AS signup_count
FROM users
WHERE YEAR(signup_date) = 2024
  AND MONTH(signup_date) BETWEEN 1 AND 6
GROUP BY MONTH(signup_date), MONTHNAME(signup_date)
ORDER BY month_number;

SELECT
    MONTHNAME(signup_date) AS month,
    COUNT(*) AS signup_count
FROM users
WHERE YEAR(signup_date) = 2024
  AND MONTH(signup_date) BETWEEN 1 AND 6
GROUP BY MONTH(signup_date), MONTHNAME(signup_date)
ORDER BY signup_count DESC
LIMIT 1;
```

Q3 — Device Analytics

```
SELECT
    device_type,
    COUNT(*) AS total_sessions,
    SUM(watch_minutes) AS total_watch_minutes,
    ROUND(AVG(watch_minutes), 2) AS average_watch_minutes,
    ROUND(
        SUM(CASE WHEN completed = 1 THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    ) AS completion_rate
FROM watch_sessions
WHERE user_id IS NOT NULL
GROUP BY device_type
ORDER BY device_type;
```

Q4 — Rating Distribution

```
SELECT
    stars,
    COUNT(*) AS rating_count,
    ROUND(COUNT(*) * 100.0 /
          (SELECT COUNT(*) FROM ratings), 2) AS percentage
FROM ratings
GROUP BY stars
ORDER BY stars;

SELECT
    ROUND(
        SUM(CASE WHEN stars IN (4, 5) THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    ) AS percentage_4_or_5_stars
FROM ratings;
```

Q5 — Originals vs Acquired

```
SELECT
    CASE
        WHEN is_original = 1 THEN 'BingePlay Original'
        ELSE 'Acquired Content'
    END AS content_type,
    COUNT(*) AS number_of_shows,
    ROUND(AVG(imdb_rating), 2) AS average_imdb_rating,
    ROUND(AVG(release_year), 2) AS average_release_year
FROM shows
GROUP BY is_original
ORDER BY is_original DESC;

SELECT
    ROUND(AVG(CASE WHEN is_original = 1 THEN imdb_rating END), 2)
        AS original_rating,
    ROUND(AVG(CASE WHEN is_original = 0 THEN imdb_rating END), 2)
        AS acquired_rating,
    ROUND(
        AVG(CASE WHEN is_original = 1 THEN imdb_rating END)
        - AVG(CASE WHEN is_original = 0 THEN imdb_rating END), 2
    ) AS rating_difference
FROM shows;
```

Q6 — Binge Day Detection

```
SELECT
    COUNT(*) AS total_binge_days
FROM (
    SELECT user_id, show_id, session_date
    FROM watch_sessions
    WHERE session_date BETWEEN '2024-04-01' AND '2024-06-30'
      AND user_id IS NOT NULL
    GROUP BY user_id, show_id, session_date
    HAVING COUNT(*) >= 5
) AS binge_days;

SELECT
    user_id,
    COUNT(*) AS binge_days
FROM (
    SELECT user_id, show_id, session_date
    FROM watch_sessions
    WHERE session_date BETWEEN '2024-04-01' AND '2024-06-30'
      AND user_id IS NOT NULL
    GROUP BY user_id, show_id, session_date
    HAVING COUNT(*) >= 5
) AS binge_days
GROUP BY user_id
ORDER BY binge_days DESC
LIMIT 1;
```

Q7 — Q1 Signups Who Never Watched

```
SELECT
    COUNT(*) AS never_watched
FROM users u
LEFT JOIN watch_sessions ws ON u.user_id = ws.user_id
WHERE u.signup_date BETWEEN '2024-01-01' AND '2024-03-31'
```

```
  AND ws.session_id IS NULL;

SELECT
    COUNT(*) AS total_q1_signups
FROM users
WHERE signup_date BETWEEN '2024-01-01' AND '2024-03-31';
```

Q8 — Over-Paying Premium/Family Users

```
WITH current_plan AS (
    SELECT
        user_id,
        plan,
        start_date,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY start_date DESC
        ) AS rn
    FROM subscriptions
    WHERE start_date <= '2024-06-30'
      AND (end_date IS NULL OR end_date > '2024-06-30')
)
SELECT COUNT(*) AS overpaying_users
FROM current_plan cp
WHERE cp.rn = 1
  AND cp.plan IN ('Premium', 'Family')
  AND NOT EXISTS (
        SELECT 1
        FROM watch_sessions ws
        JOIN shows s ON ws.show_id = s.show_id
        WHERE ws.user_id = cp.user_id
          AND s.min_plan IN ('Premium', 'Family')
    );
```

Q9 — Upgrade Success Cohort

```
WITH ordered_subscriptions AS (
    SELECT s.*,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY start_date, subscription_id
        ) AS rn
    FROM subscriptions s
),
first_basic AS (
    SELECT user_id, start_date AS basic_start_date
    FROM ordered_subscriptions
    WHERE rn = 1 AND plan = 'Basic'
),
first_upgrade AS (
    SELECT fb.user_id, MIN(os.start_date) AS upgrade_date
    FROM first_basic fb
    JOIN ordered_subscriptions os ON fb.user_id = os.user_id
    WHERE os.start_date > fb.basic_start_date
      AND os.plan IN ('Premium', 'Family')
    GROUP BY fb.user_id
),
active_users AS (
    SELECT DISTINCT user_id
    FROM subscriptions
    WHERE status = 'active'
      AND (end_date IS NULL OR end_date > '2024-06-30')
)
SELECT
    COUNT(*) AS upgrade_users,
    ROUND(AVG(DATEDIFF(fu.upgrade_date, u.signup_date)), 2)
        AS average_days_to_upgrade
FROM first_upgrade fu
JOIN users u ON fu.user_id = u.user_id
JOIN active_users au ON fu.user_id = au.user_id
WHERE u.signup_date BETWEEN '2024-01-01' AND '2024-01-31';
```

Q10 — Cliffhanger Comebacks

```
WITH comeback_events AS (
    SELECT DISTINCT
        ws1.user_id,
        ws1.show_id,
        ws1.session_date AS incomplete_date
```

```
    FROM watch_sessions ws1
    JOIN watch_sessions ws2
      ON ws1.user_id = ws2.user_id
     AND ws1.show_id = ws2.show_id
     AND ws2.session_date BETWEEN
         DATE_ADD(ws1.session_date, INTERVAL 1 DAY)
         AND DATE_ADD(ws1.session_date, INTERVAL 7 DAY)
    WHERE ws1.completed = 0
      AND ws1.user_id IS NOT NULL
)
SELECT COUNT(*) AS total_cliffhanger_comebacks
FROM comeback_events;

WITH comeback_events AS (
    SELECT DISTINCT
        ws1.user_id, ws1.show_id, ws1.session_date AS incomplete_date
    FROM watch_sessions ws1
    JOIN watch_sessions ws2
      ON ws1.user_id = ws2.user_id
     AND ws1.show_id = ws2.show_id
     AND ws2.session_date BETWEEN
         DATE_ADD(ws1.session_date, INTERVAL 1 DAY)
         AND DATE_ADD(ws1.session_date, INTERVAL 7 DAY)
    WHERE ws1.completed = 0
      AND ws1.user_id IS NOT NULL
)
SELECT ce.show_id, s.title, COUNT(*) AS comeback_count
FROM comeback_events ce
JOIN shows s ON ce.show_id = s.show_id
GROUP BY ce.show_id, s.title
ORDER BY comeback_count DESC
LIMIT 1;
```

Q11 — Consecutive-Week Engagement

```
WITH weekly_activity AS (
    SELECT DISTINCT user_id, YEARWEEK(session_date, 3) AS week_key
    FROM watch_sessions
    WHERE user_id IS NOT NULL
),
numbered_weeks AS (
    SELECT user_id, week_key,
        ROW_NUMBER() OVER (
            PARTITION BY user_id ORDER BY week_key
        ) AS rn
    FROM weekly_activity
),
week_groups AS (
    SELECT user_id, week_key, week_key - rn AS grp
    FROM numbered_weeks
),
streaks AS (
    SELECT user_id, grp, COUNT(*) AS streak_length
    FROM week_groups
    GROUP BY user_id, grp
)
SELECT
    COUNT(DISTINCT CASE WHEN streak_length >= 4 THEN user_id END)
        AS users_with_4_plus_week_streak,
    MAX(streak_length) AS longest_streak_weeks
FROM streaks;

WITH weekly_activity AS (
    SELECT DISTINCT user_id, YEARWEEK(session_date, 3) AS week_key
    FROM watch_sessions
    WHERE user_id IS NOT NULL
),
numbered_weeks AS (
    SELECT user_id, week_key,
        ROW_NUMBER() OVER (
            PARTITION BY user_id ORDER BY week_key
        ) AS rn
    FROM weekly_activity
),
week_groups AS (
    SELECT user_id, week_key - rn AS grp
    FROM numbered_weeks
),
streaks AS (
    SELECT user_id, grp, COUNT(*) AS streak_length
    FROM week_groups
```

```
    GROUP BY user_id, grp
)
SELECT user_id, streak_length
FROM streaks
ORDER BY streak_length DESC, user_id
LIMIT 1;
```

Q12 — Churn Signal Detection

```
WITH monthly_watch AS (
    SELECT
        user_id,
        SUM(CASE
            WHEN session_date BETWEEN '2024-05-01' AND '2024-05-31'
            THEN watch_minutes ELSE 0 END) AS may_minutes,
        SUM(CASE
            WHEN session_date BETWEEN '2024-06-01' AND '2024-06-30'
            THEN watch_minutes ELSE 0 END) AS june_minutes
    FROM watch_sessions
    WHERE user_id IS NOT NULL
      AND session_date BETWEEN '2024-05-01' AND '2024-06-30'
    GROUP BY user_id
)
SELECT
    mw.user_id,
    u.name,
    mw.may_minutes AS may_watch_minutes,
    mw.june_minutes AS june_watch_minutes,
    ROUND(
        (mw.may_minutes - mw.june_minutes)
        * 100.0 / mw.may_minutes, 2
    ) AS drop_percentage
FROM monthly_watch mw
JOIN users u ON mw.user_id = u.user_id
WHERE mw.may_minutes > 0
  AND mw.june_minutes <= mw.may_minutes * 0.5
ORDER BY drop_percentage DESC;

WITH monthly_watch AS (
    SELECT
        user_id,
        SUM(CASE
            WHEN session_date BETWEEN '2024-05-01' AND '2024-05-31'
            THEN watch_minutes ELSE 0 END) AS may_minutes,
        SUM(CASE
            WHEN session_date BETWEEN '2024-06-01' AND '2024-06-30'
            THEN watch_minutes ELSE 0 END) AS june_minutes
    FROM watch_sessions
    WHERE user_id IS NOT NULL
      AND session_date BETWEEN '2024-05-01' AND '2024-06-30'
    GROUP BY user_id
)
SELECT COUNT(*) AS total_churn_signal_users
FROM monthly_watch
WHERE may_minutes > 0
  AND june_minutes <= may_minutes * 0.5;
```

Project Notes

Run the queries in MySQL Workbench against the provided BingePlay database. Use the returned values as the dataset-specific answers. The SQL covers all 12 questions and includes the required advanced SQL techniques.